

Uniwersytet Mikołaja Kopernika w Toruniu
Wydział Matematyki i Informatyki

Norbert Jaskulski
nr albumu: 313097
kierunek: informatyka

Praca magisterska

Platforma do analizy i generowania struktur formalnych z wykorzystaniem Sztucznej Inteligencji

Opiekun pracy dyplomowej
dr Anna Kwiatkowska, prof.
UMK

Toruń 2026

Spis treści

1	Wprowadzenie	3
1.1	Kontekst i motywacja	3
1.2	Cel i zakres pracy	3
1.3	Hipoteza badawcza	4
1.4	Zakres pracy i ograniczenia	5
1.5	Struktura pracy	5
2	Podstawy teoretyczne języków formalnych oraz metod ich analizy i generowania	6
2.1	Wprowadzenie	6
2.2	Zastosowanie języków bezkontekstowych w analizie składniowej	6
2.3	Podstawowe pojęcia języków formalnych	7
2.4	Operacje na językach formalnych	7
2.5	Gramatyki formalne	8
2.6	Wyrażenia regularne	8
2.7	Automaty skończone	9
2.8	Języki bezkontekstowe i gramatyki bezkontekstowe	9
2.9	Automaty ze stosem	11
2.10	Algorytm L* Angluina i uczenie języków formalnych	12
2.11	Analiza i generowanie języków formalnych	13
3	Metody analizy i generowania struktur formalnych z wykorzystaniem sztucznej inteligencji	15
3.1	Wprowadzenie	15
3.2	Zastosowanie sztucznej inteligencji w językach formalnych . .	16
3.3	Modele językowe i generatywna sztuczna inteligencja	18
3.4	Automatyczne uczenie automatów i gramatyk	19

3.5	Rozszerzenie algorytmu L^* dla automatów ze stosem	21
3.6	Założenia projektowe platformy badawczej	23

1 Wprowadzenie

1.1 Kontekst i motywacja

Rozwój technologii sztucznej inteligencji w ostatniej dekadzie diametralnie zmienił sposoby, w jakie przetwarzamy i interpretujemy dane językowe. Szczególnie przełomowa okazała się architektura transformera, wprowadzona w 2017 roku, która zrewolucjonizowała obszar przetwarzania języka naturalnego (NLP). Na jej podstawie powstały tzw. duże modele językowe (LLM, *Large Language Models*), takie jak GPT, Gemini czy Copilot, osiągające wysoki poziom kompetencji w rozumieniu i generowaniu tekstu w języku naturalnym.

Mimo znakomitych wyników w zadaniach związanych z językiem naturalnym, zdolność LLM do operowania na strukturach języków formalnych pozostaje stosunkowo słabo zbadana. Języki formalne, stanowiące fundament m.in. kompilatorów, automatów skończonych, gramatyk formalnych czy opisu protokołów, wymagają ścisłych reguł składniowych i semantycznych, których przestrzeganie jest kluczowe dla poprawności tworzonych struktur. Integracja LLM z takimi domenami może przynieść korzyści w automatycznym projektowaniu i analizie formalnych modeli systemów, a także w dydaktyce informatyki teoretycznej i języków formalnych.

Pojawia się jednak wyzwanie: LLM są trenowane głównie na danych tekstowych w języku naturalnym, co nie gwarantuje ich zdolności do ścisłego przestrzegania reguł syntaktycznych języków formalnych. Badanie w jaki sposób można przystosować modele tego typu do interpretacji i generowania struktur formalnych, ma zatem istotne znaczenie zarówno teoretyczne, jak i praktyczne.

1.2 Cel i zakres pracy

Głównym celem niniejszej pracy jest opracowanie platformy do analizy i generowania struktur języków formalnych z wykorzystaniem sztucznej inteligencji, a w szczególności zbadanie zdolności dużych modeli językowych do transformacji tekstu w języku naturalnym na reprezentacje języków formalnych.

Praca ma charakter badawczo-rozwojowy i obejmuje dwa główne obszary:

- **Badawczy** - eksperymentalne sprawdzenie możliwości LLM w rozwiązywaniu zadań związanych z językami formalnymi, takich jak generowanie automatów skończonych, gramatyk bezkontekstowych czy drzew decyzyjnych na podstawie opisów w języku naturalnym.
- **Rozwojowy** - implementacja platformy programistycznej, która łączy możliwości LLM z klasycznymi metodami analizy struktur formalnych (np. algorytm L^* , metoda DCSA), umożliwiając ich wizualizację, weryfikację i eksperymentalną ocenę.

Dodatkowym celem jest ocena skuteczności różnych podejść do adaptacji LLM od *prompt engineeringu* po lekkie metody strojenia (np. LoRA) w kontekście poprawności i spójności generowanych struktur formalnych.

1.3 Hipoteza badawcza

Hipoteza przyjęta w pracy zakłada, że:

Odpowiednio przystosowane duże modele językowe, w szczególności transformatory, mogą efektywnie interpretować opisy w języku naturalnym i generować struktury języków formalnych porównywalne pod względem poprawności z konstrukcjami uzyskiwanymi metodami klasycznymi.

Sprawdzenie tej hipotezy wymaga zaprojektowania eksperymentów porównawczych, które pozwolą ocenić:

- poprawność syntaktyczną i semantyczną generowanych struktur,
- wierność odwzorowania opisu w języku naturalnym,
- efektywność i stabilność transformacji przy różnych typach zadań i poziomach skomplikowania struktur.

1.4 Zakres pracy i ograniczenia

Zakres badań obejmuje wybrane klasy języków formalnych, przede wszystkim języki regularne (reprezentowane automatami skończonymi) oraz bezkontekstowe (CFG, *Context-Free Grammars*). Wynika to z ich fundamentalnego znaczenia w informatyce teoretycznej i przetwarzaniu języków.

Praca nie koncentruje się na trenowaniu modeli od podstaw, lecz na eksperymentach z istniejącymi LLM oraz na badaniu ich adaptacji do zadań strukturalnych. Ograniczenia wynikają również z dostępnych zasobów obliczeniowych, dlatego stosowane będą głównie modele typu open-source (np. LLaMA 2), które można dostosować do specyficznych zadań.

1.5 Struktura pracy

Praca składa się z sześciu rozdziałów.

- W rozdziale 2 przedstawiono teoretyczne podstawy działania dużych modeli językowych i ich roli w przetwarzaniu języka naturalnego.
- Rozdział 3 opisuje proces transformacji języka naturalnego do języków formalnych, uwzględniając istniejące metody i potencjalne problemy.
- Rozdział 4 omawia projekt i implementację platformy badawczej, czyli narzędzia do analizy i generowania struktur formalnych z wykorzystaniem AI.
- W rozdziale 5 zaprezentowano wyniki eksperymentów przeprowadzonych z użyciem platformy, w tym ocenę skuteczności LLM w generowaniu struktur formalnych.
- Rozdział 6 zawiera podsumowanie i wnioski, w tym ocenę przyjętej hipotezy badawczej oraz wskazanie dalszych kierunków rozwoju.

2 Podstawy teoretyczne języków formalnych oraz metod ich analizy i generowania

2.1 Wprowadzenie

Języki formalne stanowią jeden z fundamentalnych obszarów informatyki teoretycznej, będąc podstawą opisu procesów obliczeniowych oraz struktur symbolicznych. Ich znaczenie wynika z możliwości precyzyjnego definiowania zbiorów wyrażeń oraz jednoznacznej interpretacji reguł ich budowy. W przeciwieństwie do języków naturalnych, które cechują się wieloznacznością i zależnością od kontekstu, języki formalne opierają się na ścisłych zasadach matematycznych.

W niniejszym rozdziale przedstawione zostaną podstawowe definicje oraz modele matematyczne, które stanowią fundament dla projektowanej platformy służącej do analizy i generowania struktur języków formalnych z wykorzystaniem sztucznej inteligencji.

2.2 Zastosowanie języków bezkontekstowych w analizie składniowej

Języki bezkontekstowe odgrywają kluczową rolę w analizie składniowej, szczególnie w kontekście języków programowania. Struktury takie jak zagnieżdżone wyrażenia, bloki kodu czy wywołania funkcji mogą być naturalnie modelowane przy użyciu gramatyk bezkontekstowych.

W praktyce stosuje się różne algorytmy parsowania, takie jak parsery LL oraz LR, które umożliwiają efektywną analizę składniową na podstawie gramatyk bezkontekstowych. Proces ten jest niezbędny do weryfikacji poprawności składniowej programów oraz budowy ich reprezentacji wewnętrznej.

W kontekście projektowanej platformy języki bezkontekstowe oraz automaty ze stosem umożliwiają analizę bardziej złożonych struktur formalnych niż w przypadku języków regularnych, co znacząco rozszerza możliwości systemu.

2.3 Podstawowe pojęcia języków formalnych

Podstawą każdego języka formalnego jest alfabet, definiowany jako skończony zbiór symboli:

$$\Sigma = \{a_1, a_2, \dots, a_n\}, \quad \Sigma \neq \emptyset \quad (1)$$

Na bazie alfabetu definiuje się słowa, czyli skończone ciągi symboli należących do alfabetu. Długość słowa w oznaczamy jako $|w|$. Szczególnym przypadkiem jest słowo puste:

$$\varepsilon, \quad |\varepsilon| = 0 \quad (2)$$

Zbiór wszystkich możliwych słów nad alfabetem Σ definiujemy jako:

$$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i \quad (3)$$

gdzie:

$$\Sigma^0 = \{\varepsilon\}, \quad \Sigma^1 = \Sigma, \quad \Sigma^2 = \Sigma \cdot \Sigma \quad (4)$$

Język formalny definiuje się jako dowolny podzbiór zbioru Σ^* :

$$L \subseteq \Sigma^* \quad (5)$$

2.4 Operacje na językach formalnych

Języki formalne można traktować jako zbiory, dlatego można na nich wykonywać różne operacje. Suma języków L_1 i L_2 definiowana jest jako:

$$L_1 \cup L_2 = \{w \mid w \in L_1 \vee w \in L_2\} \quad (6)$$

Konkatenacja języków ma postać:

$$L_1 \cdot L_2 = \{xy \mid x \in L_1, y \in L_2\} \quad (7)$$

Potęgowanie języka definiujemy jako:

$$L^n = \underbrace{L \cdot L \cdot \dots \cdot L}_{n \text{ razy}}, \quad L^0 = \{\varepsilon\} \quad (8)$$

Na tej podstawie definiujemy domknięcie Kleene'ego:

$$L^* = \bigcup_{n=0}^{\infty} L^n \quad (9)$$

oraz domknięcie dodatnie:

$$L^+ = \bigcup_{n=1}^{\infty} L^n \quad (10)$$

2.5 Gramatyki formalne

Gramatyka formalna jest narzędziem generowania języków i definiowana jest jako:

$$G = (V, \Sigma, P, S) \quad (11)$$

gdzie V oznacza zbiór symboli nieterminalnych, Σ zbiór symboli terminalnych, P zbiór produkcji, a $S \in V$ symbol startowy.

Produkcje mają postać:

$$\alpha \rightarrow \beta \quad (12)$$

gdzie $\alpha \in (V \cup \Sigma)^+$ oraz $\beta \in (V \cup \Sigma)^*$.

Proces generowania słowa opisujemy jako:

$$S \Rightarrow^* w, \quad w \in \Sigma^* \quad (13)$$

gdzie $\Rightarrow^*$ oznacza domknięcie przechodnie relacji wyprowadzenia.

2.6 Wyrażenia regularne

Wyrażenia regularne stanowią formalny sposób opisu języków regularnych. Definiuje się je rekurencyjnie:

- $\emptyset$ jest wyrażeniem regularnym,
- ε jest wyrażeniem regularnym,
- dla każdego $a \in \Sigma$, symbol a jest wyrażeniem regularnym.

Jeżeli r i s są wyrażeniami regularnymi, to:

$$r + s, \quad rs, \quad r^* \quad (14)$$

są również wyrażeniami regularnymi.

Język opisany przez wyrażenie regularne r oznaczamy jako $L(r)$.

2.7 Automaty skończone

Automat skończony definiuje się jako:

$$M = (Q, \Sigma, \delta, q_0, F) \quad (15)$$

gdzie:

- Q jest zbiorem stanów,
- Σ alfabetem,
- $\delta : Q \times \Sigma \rightarrow Q$ funkcją przejścia,
- $q_0 \in Q$ stanem początkowym,
- $F \subseteq Q$ zbiorem stanów akceptujących.

Dla automatów niedeterministycznych funkcja przejścia ma postać:

$$\delta : Q \times \Sigma \rightarrow 2^Q \quad (16)$$

Słowo w jest akceptowane przez automat, jeśli:

$$\hat{\delta}(q_0, w) \in F \quad (17)$$

2.8 Języki bezkontekstowe i gramatyki bezkontekstowe

Języki bezkontekstowe stanowią istotną klasę języków formalnych, szczególnie ważną w kontekście analizy składniowej języków programowania. Język

bezkontekstowy jest definiowany jako język generowany przez gramatykę bezkontekstową.

Gramatyka bezkontekstowa (CFG – *Context-Free Grammar*) jest szczególnym przypadkiem gramatyki formalnej, w której wszystkie produkcje mają postać:

$$A \rightarrow \gamma \quad (18)$$

gdzie $A \in V$ jest pojedynczym symbolem nieterminalnym, a $\gamma \in (V \cup \Sigma)^*$ jest dowolnym ciągiem symboli terminalnych i nieterminalnych.

Język generowany przez gramatykę bezkontekstową definiuje się jako:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\} \quad (19)$$

Gramatyki bezkontekstowe umożliwiają opis struktur zagnieżdżonych, takich jak poprawnie zbalansowane nawiasy. Przykładowa gramatyka generująca taki język może być zapisana jako:

$$S \rightarrow (S)S \mid \varepsilon \quad (20)$$

Istotnym elementem związanym z gramatykami bezkontekstowymi jest możliwość reprezentacji procesu generowania w postaci drzewa składniowego, które obrazuje strukturę wyprowadzenia słowa. Węzły drzewa odpowiadają symbolom nieterminalnym, natomiast liście symbolom terminalnym.

W analizie języków bezkontekstowych często wykorzystuje się specjalne postacie gramatyk, które upraszczają ich przetwarzanie. Jedną z najważniejszych jest postać normalna Chomsky'ego (CNF – *Chomsky Normal Form*), w której każda produkcja ma jedną z postaci:

$$A \rightarrow BC \quad \text{lub} \quad A \rightarrow a \quad (21)$$

gdzie $A, B, C \in V$, a $a \in \Sigma$.

Inną istotną postacią jest postać normalna Greibacha, w której produkcje mają postać:

$$A \rightarrow a\alpha \quad (22)$$

gdzie $a \in \Sigma$, a $\alpha \in V^*$.

Normalne formy gramatyk są szczególnie przydatne w konstrukcji algorytmów parsujących oraz dowodach własności języków.

2.9 Automaty ze stosem

Automaty ze stosem (PDA – *Pushdown Automata*) stanowią model obliczeniowy zdolny do rozpoznawania języków bezkontekstowych. Rozszerzają one możliwości automatów skończonych poprzez wprowadzenie dodatkowej pamięci w postaci stosu.

Formalnie automat ze stosem definiuje się jako siódmkę:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F) \quad (23)$$

gdzie:

- Q jest zbiorem stanów,
- Σ alfabetem wejściowym,
- Γ alfabetem stosu,
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow 2^{Q \times \Gamma^*}$ funkcją przejścia,
- $q_0 \in Q$ stanem początkowym,
- $Z_0 \in \Gamma$ symbolem początkowym stosu,
- $F \subseteq Q$ zbiorem stanów akceptujących.

Działanie automatu ze stosem polega na przetwarzaniu symboli wejściowych oraz manipulowaniu zawartością stosu poprzez operacje dodawania (push) i usuwania (pop) symboli.

Automat ze stosem akceptuje słowo w , jeśli istnieje ciąg przejść prowadzący od konfiguracji początkowej do konfiguracji akceptującej. Formalnie można to zapisać jako:

$$(q_0, w, Z_0) \Rightarrow^* (q, \varepsilon, \gamma) \quad (24)$$

gdzie $q \in F$.

Alternatywnie dopuszcza się akceptację przez pusty stos:

$$(q_0, w, Z_0) \Rightarrow^* (q, \varepsilon, \varepsilon) \quad (25)$$

Automaty ze stosem są ściśle powiązane z gramatykami bezkontekstowymi. Istnieje fundamentalne twierdzenie, które mówi, że dla każdego języka bezkontekstowego istnieje równoważny automat ze stosem oraz odwrotnie.

2.10 Algorytm L* Angluin i uczenie języków formalnych

Jednym z istotnych zagadnień w kontekście analizy i generowania języków formalnych jest problem ich automatycznego uczenia się na podstawie obserwacji. Klasycznym podejściem do tego problemu jest algorytm L* zaproponowany przez Danę Angluin w 1987 roku. Algorytm ten umożliwia identyfikację języka regularnego poprzez konstruowanie minimalnego deterministycznego automatu skończonego na podstawie interakcji z tzw. nauczycielem (*teacher*).

Model uczenia wykorzystywany w algorytmie L* opiera się na dwóch typach zapytań: zapytaniach o przynależność oraz zapytaniach równoważności. Zapytanie o przynależność polega na sprawdzeniu, czy dane słowo w należy do języka L , co można formalnie zapisać jako:

$$\text{MQ}(w) = \begin{cases} 1, & \text{gdy } w \in L \\ 0, & \text{gdy } w \notin L \end{cases} \quad (26)$$

Zapytanie równoważności polega natomiast na sprawdzeniu, czy zaproponowany przez uczący się model automat M jest równoważny językowi docelowemu L , czyli:

$$L(M) = L \quad (27)$$

W przypadku, gdy automat nie jest równoważny, nauczyciel zwraca kontrprzykład w , taki że:

$$w \in L(M) \oplus L \quad (28)$$

gdzie $\oplus$ oznacza różnicę symetryczną języków.

Algorytm L* opiera się na strukturze zwanej tabelą obserwacji (*observation table*), która jest definiowana jako trójka:

$$(S, E, T) \quad (29)$$

gdzie $S \subseteq \Sigma^*$ jest zbiorem prefiksów, $E \subseteq \Sigma^*$ zbiorem sufixów, a funkcja $T : (S \cup S\Sigma) \times E \rightarrow \{0, 1\}$ określa przynależność słów do języka:

$$T(s, e) = \begin{cases} 1, & \text{gdy } se \in L \\ 0, & \text{gdy } se \notin L \end{cases} \quad (30)$$

Tabela obserwacji musi spełniać dwa kluczowe warunki: domknięcia oraz spójności. Warunek domknięcia oznacza, że dla każdego $t \in S\Sigma$ istnieje taki $s \in S$, że wiersze odpowiadające t i s są identyczne:

$$\forall t \in S\Sigma \quad \exists s \in S \quad \text{row}(t) = \text{row}(s) \quad (31)$$

Warunek spójności zapewnia, że jeśli dwa wiersze są identyczne, to ich rozszerzenia o dowolny symbol również pozostają zgodne:

$$\text{row}(s_1) = \text{row}(s_2) \Rightarrow \forall a \in \Sigma \quad \text{row}(s_1 a) = \text{row}(s_2 a) \quad (32)$$

Po spełnieniu tych warunków możliwe jest skonstruowanie deterministycznego automatu skończonego:

$$M = (Q, \Sigma, \delta, q_0, F) \quad (33)$$

gdzie zbiór stanów Q odpowiada unikalnym wierszom tabeli obserwacji.

Algorytm działa iteracyjnie, rozszerzając zbiory S i E oraz aktualizując tabelę obserwacji aż do momentu uzyskania poprawnego automatu. W przypadku otrzymania kontrprzykładu tabela jest aktualizowana poprzez dodanie odpowiednich prefiksów.

Złożoność algorytmu L^* jest wielomianowa względem liczby stanów minimalnego automatu oraz długości kontrprzykładów, co czyni go efektywnym narzędziem w praktyce.

Algorytm L^* znajduje zastosowanie w wielu obszarach, takich jak inżynieria odwrotna protokołów, testowanie systemów, weryfikacja formalna oraz uczenie modeli zachowań systemów. W kontekście projektowanej platformy algorytm ten może być wykorzystany do automatycznego wnioskowania struktur formalnych na podstawie przykładów dostarczonych przez użytkownika, co stanowi istotny element integracji metod sztucznej inteligencji z teorią języków formalnych.

2.11 Analiza i generowanie języków formalnych

Analiza języka formalnego polega na sprawdzeniu, czy dane słowo należy do języka:

$$w \in L \quad (34)$$

W przypadku gramatyk:

$$S \Rightarrow^* w \quad (35)$$

Parsowanie polega na budowie drzewa składniowego reprezentującego proces wyprowadzenia.

Generowanie języka polega na wyznaczeniu zbioru:

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\} \quad (36)$$

Proces ten może być realizowany poprzez systematyczne stosowanie reguł produkcji.

W kontekście współczesnych systemów informatycznych istotne znaczenie ma zastosowanie sztucznej inteligencji w analizie i generowaniu języków formalnych. Można to ująć jako odwzorowanie:

$$f : NL \rightarrow FL \quad (37)$$

gdzie NL oznacza język naturalny, a FL język formalny.

3 Metody analizy i generowania struktur formalnych z wykorzystaniem sztucznej inteligencji

3.1 Wprowadzenie

Dynamiczny rozwój metod sztucznej inteligencji oraz uczenia maszynowego w ostatnich latach znacząco wpłynął na sposoby analizy, modelowania i generowania struktur formalnych. Szczególne znaczenie zyskały techniki umożliwiające automatyczne rozpoznawanie wzorców, inferencję gramatyk oraz uczenie modeli reprezentujących języki formalne. Rozwiązania tego typu znajdują zastosowanie między innymi w kompilatorach, analizatorach składniowych, systemach przetwarzania języka naturalnego, weryfikacji formalnej oprogramowania oraz systemach automatycznej analizy kodu źródłowego.

Klasyczne metody analizy języków formalnych opierają się głównie na deterministycznych modelach matematycznych, takich jak automaty skończone, automaty ze stosem czy gramatyki formalne. Podejścia te zapewniają wysoką precyzję opisu języków, jednak w przypadku bardziej złożonych struktur formalnych ich projektowanie i analiza mogą być czasochłonne oraz trudne do automatyzacji. Z tego względu coraz większe znaczenie zyskują metody wykorzystujące sztuczną inteligencję do wspomagania procesu generowania oraz analizy struktur formalnych.

Istotnym obszarem badań jest problem inferencji języków formalnych, czyli automatycznego uczenia modeli opisujących dany język na podstawie przykładów lub interakcji z nauczycielem. Jednym z najbardziej znanych algorytmów w tym zakresie jest algorytm L^* zaproponowany przez Danę Angluin, umożliwiający aktywne uczenie automatów skończonych. Algorytm ten stanowi podstawę wielu współczesnych metod automatycznej inferencji struktur formalnych. Jego klasyczna wersja ogranicza się jednak głównie do języków regularnych reprezentowanych przez automaty skończone.

W przypadku języków bezkontekstowych oraz struktur wymagających pamięci stosowej konieczne staje się zastosowanie bardziej zaawansowanych modeli, takich jak automaty ze stosem. Rozszerzenie metod aktywnego uczenia na tego rodzaju struktury stanowi obecnie istotny problem badawczy, wymagający opracowania nowych mechanizmów reprezentacji oraz analizy stanów

i przejść.

Celem niniejszego rozdziału jest przedstawienie współczesnych metod analizy i generowania struktur formalnych z wykorzystaniem sztucznej inteligencji, ze szczególnym uwzględnieniem problemu uczenia automatów i gramatyk formalnych. Omówione zostaną zarówno klasyczne podejścia inferencyjne, jak i nowoczesne metody wykorzystujące modele generatywne oraz uczenie aktywne. Rozdział stanowi również podstawę teoretyczną dla opracowanej w ramach pracy platformy badawczej oraz zaproponowanego rozszerzenia algorytmu L^* przeznaczonego dla automatów ze stosem.

3.2 Zastosowanie sztucznej inteligencji w językach formalnych

Sztuczna inteligencja odgrywa coraz większą rolę w analizie, rozpoznawaniu oraz generowaniu języków formalnych. Współczesne systemy informatyczne wykorzystują metody uczenia maszynowego i sztucznej inteligencji do automatyzacji procesów związanych z analizą składniową, inferencją gramatyk, modelowaniem struktur formalnych oraz weryfikacją poprawności składniowej danych wejściowych. Rozwiązania tego typu znajdują zastosowanie zarówno w klasycznych systemach kompilatorów, jak i nowoczesnych systemach przetwarzania języka naturalnego oraz automatycznej analizy kodu źródłowego.

Jednym z głównych obszarów zastosowań sztucznej inteligencji w językach formalnych jest problem inferencji gramatyk formalnych. Inferencja polega na automatycznym tworzeniu modelu języka na podstawie dostarczonych przykładów lub interakcji z systemem uczącym. W praktyce umożliwia to budowę automatów oraz gramatyk opisujących zachowanie analizowanego systemu bez konieczności ręcznego definiowania wszystkich reguł produkcji i przejść stanów.

Istotne znaczenie mają również metody aktywnego uczenia automatów (*active automata learning*), w których system uczący zadaje pytania dotyczące przynależności słów do języka oraz wykorzystuje kontrprzykłady do stopniowego budowania modelu formalnego. Podejście to zostało spopularyzowane między innymi przez algorytm L^* zaproponowany przez Danę Angluin. Algorytm ten umożliwia efektywne uczenie automatów skończonych reprezen-

tujących języki regularne i stanowi podstawę wielu współczesnych systemów inferencji języków formalnych.

W ostatnich latach znaczący rozwój osiągnęły również modele generatywnej sztucznej inteligencji, w szczególności modele oparte na architekturze transformera. Systemy te potrafią generować teksty, fragmenty kodu źródłowego oraz struktury składniowe na podstawie wzorców wykrytych w dużych zbiorach danych treningowych. Chociaż modele generatywne charakteryzują się dużą skutecznością w analizie i generowaniu danych tekstowych, często nie gwarantują pełnej zgodności z formalnymi regułami składniowymi. Z tego względu coraz częściej podejmowane są próby łączenia metod symbolicznych z technikami uczenia maszynowego, tworząc podejścia hybrydowe określane jako systemy neuro-symboliczne.

Sztuczna inteligencja znajduje zastosowanie także w automatycznej analizie kodu źródłowego, wykrywaniu błędów składniowych, generowaniu parserów oraz optymalizacji procesów kompilacji. W systemach tych języki formalne wykorzystywane są do reprezentacji składni programów, natomiast algorytmy uczenia maszynowego wspomagają analizę wzorców oraz automatyczne wykrywanie nieprawidłowości.

Szczególnie istotnym kierunkiem badań jest obecnie rozszerzenie metod aktywnego uczenia na języki bezkontekstowe oraz automaty ze stosem. W przeciwieństwie do automatów skończonych modele te umożliwiają reprezentację bardziej złożonych struktur składniowych wymagających pamięci stosowej, takich jak zagnieżdżone wyrażenia czy struktury drzewiaste. Problem efektywnego uczenia takich modeli pozostaje jednym z ważniejszych wyzwań współczesnej teorii języków formalnych oraz sztucznej inteligencji.

W niniejszej pracy sztuczna inteligencja wykorzystywana jest jako element wspomagający analizę i generowanie struktur formalnych w ramach opracowanej platformy badawczej. Szczególną uwagę poświęcono możliwości rozszerzenia klasycznego algorytmu L^* na modele wykorzystujące mechanizm stosu, co pozwala na analizę bardziej złożonych języków formalnych niż języki regularne.

3.3 Modele językowe i generatywna sztuczna inteligencja

Współczesny rozwój sztucznej inteligencji doprowadził do powstania zaawansowanych modeli językowych zdolnych do analizy, generowania oraz transformacji danych tekstowych i struktur formalnych. Szczególne znaczenie zyskały modele generatywne wykorzystujące architekturę transformera, które umożliwiły znaczące zwiększenie skuteczności systemów przetwarzania języka naturalnego (*Natural Language Processing - NLP*) oraz automatycznej analizy kodu źródłowego.

Modele językowe (*Language Models - LM*) są systemami uczonymi na dużych zbiorach danych tekstowych, których zadaniem jest przewidywanie kolejnych elementów sekwencji na podstawie wcześniej występującego kontekstu. W praktyce oznacza to możliwość generowania tekstu, uzupełniania fragmentów kodu, tłumaczenia języków naturalnych oraz rozpoznawania struktur składniowych. W zależności od architektury modele te mogą wykorzystywać sieci rekurencyjne (*Recurrent Neural Networks - RNN*), sieci LSTM (*Long Short-Term Memory*) lub współcześnie dominującą architekturę transformera.

Przełomowym momentem w rozwoju modeli językowych było zaproponowanie architektury transformera, opartej na mechanizmie uwagi (*attention mechanism*). Mechanizm ten umożliwia analizę zależności pomiędzy elementami sekwencji niezależnie od ich odległości w tekście, co pozwala na efektywniejsze modelowanie struktur składniowych oraz semantycznych. Dzięki temu możliwe stało się tworzenie dużych modeli językowych (*Large Language Models - LLM*), takich jak GPT, BERT czy LLaMA.

Generatywna sztuczna inteligencja wykorzystuje modele językowe do tworzenia nowych danych na podstawie wzorców wykrytych w procesie uczenia. W przypadku języków formalnych modele generatywne mogą być wykorzystywane do automatycznego generowania wyrażeń regularnych, gramatyk formalnych, struktur składniowych oraz fragmentów kodu źródłowego. Systemy tego typu znajdują zastosowanie między innymi w automatycznym generowaniu parserów, wspomaganiu programowania oraz analizie poprawności składniowej programów komputerowych.

Pomimo wysokiej skuteczności modeli generatywnych, ich wykorzystanie w kontekście języków formalnych wiąże się z istotnymi ograniczeniami. Modele językowe działają głównie w oparciu o zależności statystyczne występujące w

danych treningowych, co oznacza, że generowane struktury nie zawsze spełniają ściśle reguły formalne. W przeciwieństwie do klasycznych automatów i gramatyk formalnych modele generatywne nie gwarantują pełnej poprawności składniowej ani deterministycznego działania.

Problem ten jest szczególnie widoczny w przypadku języków bezkontekstowych oraz struktur wymagających zachowania poprawnych zależności zagnieźdzonych, takich jak nawiasowania czy drzewa składniowe. W takich sytuacjach modele probabilistyczne mogą generować struktury niezgodne z definicją języka formalnego, pomimo pozornie poprawnej składni powierzchniowej.

W niniejszej pracy modele sztucznej inteligencji wykorzystywane są jako element wspomagający analizę i generowanie struktur formalnych w opracowanej platformie badawczej. Szczególne znaczenie ma możliwość połączenia klasycznych metod inferencji automatów z nowoczesnymi technikami sztucznej inteligencji w celu rozszerzenia możliwości analizy języków bezkontekstowych oraz automatów ze stosem.

3.4 Automatyczne uczenie automatów i gramatyk

Automatyczne uczenie automatów i gramatyk formalnych stanowi jeden z istotnych obszarów badań z pogranicza teorii języków formalnych, sztucznej inteligencji oraz uczenia maszynowego. Celem tego procesu jest skonstruowanie modelu formalnego opisującego dany język na podstawie dostępnych danych wejściowych, takich jak przykładowe słowa należące do języka, kontrprzykłady lub odpowiedzi udzielane przez system nauczycielski. Proces ten określany jest mianem inferencji gramatyk (*grammar inference*) lub uczenia automatów (*automata learning*).

Klasyczne podejście do projektowania automatów i gramatyk formalnych wymaga ręcznego definiowania stanów, przejść oraz reguł produkcji. W przypadku złożonych języków formalnych proces ten może być trudny, czasochłonny oraz podatny na błędy. Automatyczne metody uczenia umożliwiają częściową lub całkowitą automatyzację tego procesu poprzez analizę dostarczonych danych oraz stopniowe budowanie modelu opisującego zachowanie badanego języka.

Wyróżnia się dwa podstawowe podejścia do uczenia języków formalnych:

uczenie pasywne (*passive learning*) oraz uczenie aktywne (*active learning*). W podejściu pasywnym system uczący otrzymuje zbiór przykładowych słów należących lub nienależących do języka i na ich podstawie próbuje skonstruować odpowiedni model formalny. Skuteczność tej metody zależy w dużym stopniu od jakości oraz kompletności danych treningowych.

Uczenie aktywne opiera się natomiast na interakcji pomiędzy systemem uczącym a nauczycielem (*teacher*). Algorytm może zadawać pytania dotyczące przynależności słów do języka (*membership queries*) oraz proponować hipotezy opisujące aktualny model języka. W przypadku błędnej hipotezy nauczyciel zwraca kontrprzykład (*counterexample*), który wykorzystywany jest do dalszego udoskonalania modelu.

Jednym z najbardziej znanych algorytmów aktywnego uczenia automatów jest algorytm L^* zaproponowany przez Danę Angluin. Algorytm ten umożliwia efektywne uczenie deterministycznych automatów skończonych reprezentujących języki regularne. Działanie algorytmu opiera się na budowie tablicy obserwacji (*observation table*), która przechowuje informacje o zachowaniu analizowanego języka dla różnych prefiksów oraz sufiksów słów wejściowych.

Algorytm L^* znajduje zastosowanie między innymi w analizie protokołów komunikacyjnych, testowaniu oprogramowania, inżynierii odwrótej oraz automatycznej analizie systemów reaktywnych. Jego istotną zaletą jest możliwość konstruowania modeli formalnych bez znajomości wewnętrznej implementacji badanego systemu, wyłącznie na podstawie obserwacji jego zachowania.

Pomimo dużej skuteczności klasyczny algorytm L^* posiada istotne ograniczenia wynikające z faktu, że operuje wyłącznie na językach regularnych reprezentowanych przez automaty skończone. W praktyce wiele rzeczywistych struktur formalnych, takich jak składnia języków programowania czy struktury zagnieżdżone, wymaga zastosowania języków bezkontekstowych oraz automatów ze stosem. W takich przypadkach klasyczne modele automatów skończonych okazują się niewystarczające.

Rozszerzenie metod aktywnego uczenia na automaty ze stosem oraz gramatyki bezkontekstowe stanowi obecnie jeden z ważniejszych kierunków badań w dziedzinie inferencji języków formalnych. Problem ten jest znacznie bardziej złożony ze względu na konieczność modelowania pamięci stosowej oraz zależności zagnieżdżonych pomiędzy symbolami języka.

Współczesne podejścia badawcze coraz częściej łączą klasyczne algorytmy in-

ferencji z metodami sztucznej inteligencji oraz uczenia maszynowego. Podejścia hybrydowe umożliwiają analizę bardziej złożonych struktur formalnych oraz zwiększenie skuteczności procesu uczenia modeli językowych. Szczególnie obiecujące są rozwiązania wykorzystujące połączenie metod symbolicznych z modelami neuronowymi, które pozwalają na automatyczne wykrywanie wzorców oraz adaptacyjne budowanie struktur formalnych.

W niniejszej pracy problem automatycznego uczenia automatów i gramatyk formalnych stanowi jeden z głównych obszarów badawczych. Szczególną uwagę poświęcono możliwości rozszerzenia klasycznego algorytmu L^* na modele wykorzystujące mechanizm stosu, co pozwala na analizę bardziej złożonych języków formalnych reprezentowanych przez automaty ze stosem.

3.5 Rozszerzenie algorytmu L^* dla automatów ze stosem

Klasyczny algorytm L^* zaproponowany przez Danę Angluin stanowi jedno z najważniejszych rozwiązań w dziedzinie aktywnego uczenia automatów skończonych. Algorytm ten umożliwia efektywne konstruowanie deterministycznych automatów skończonych na podstawie odpowiedzi na pytania o przynależność słów do języka oraz kontrprzykładów dostarczanych przez nauczyciela. Pomimo wysokiej skuteczności metoda ta ogranicza się jednak wyłącznie do języków regularnych, których możliwości reprezentacyjne są niewystarczające dla wielu bardziej złożonych struktur formalnych.

W praktycznych zastosowaniach często występują języki wymagające obsługi struktur zagnieżdżonych oraz zależności hierarchicznych. Przykładami takich języków są składnie języków programowania, struktury XML, wyrażenia arytmetyczne z nawiasami czy drzewa składniowe wykorzystywane w analizie języka naturalnego. Struktury tego typu należą zazwyczaj do klasy języków bezkontekstowych i wymagają zastosowania modeli posiadających pamięć stosową, takich jak automaty ze stosem (*Pushdown Automata* – *PDA*).

Automaty ze stosem rozszerzają możliwości automatów skończonych poprzez dodanie mechanizmu stosu umożliwiającego zapamiętywanie informacji o wcześniej przetwarzanych symbolach wejściowych. Dzięki temu możliwe staje się rozpoznawanie struktur zagnieżdżonych oraz zależności rekurencyjnych, które nie mogą zostać poprawnie opisane przez klasyczne automaty skoń-

zione.

Rozszerzenie algorytmu L^* na automaty ze stosem stanowi istotny problem badawczy ze względu na znaczący wzrost złożoności modelu formalnego. W przeciwieństwie do automatów skończonych, w których stan systemu opisany jest wyłącznie przez aktualny stan automatu, w automatach ze stosem konieczne jest dodatkowe uwzględnienie zawartości stosu oraz operacji wykonywanych na jego elementach. Powoduje to zwiększenie liczby możliwych konfiguracji oraz utrudnia proces inferencji modelu.

W ramach niniejszej pracy zaproponowano rozszerzenie klasycznego algorytmu L^* umożliwiające analizę struktur formalnych wykorzystujących mechanizm stosowy. Opracowane podejście bazuje na idei aktywnego uczenia poprzez zadawanie pytań dotyczących przynależności słów do języka oraz analizę kontrprzykładów, jednak dodatkowo uwzględnia informacje związane z operacjami wykonywanymi na stosie.

W proponowanym rozwiązaniu klasyczna tablica obserwacji została rozszerzona o mechanizmy umożliwiające reprezentację kontekstu stosowego oraz zależności pomiędzy symbolami wejściowymi i strukturą stosu. Dzięki temu możliwe staje się modelowanie języków zawierających konstrukcje zagnieżdżone oraz analizowanie przejść zależnych od aktualnej zawartości stosu.

Istotnym elementem opracowanego podejścia jest również analiza kontrprzykładów w kontekście zmian zachodzących na stosie automatu. W klasycznym algorytmie L^* kontrprzykłady wykorzystywane są głównie do rozróżniania stanów automatu skończonego. W przypadku automatów ze stosem konieczne staje się dodatkowe uwzględnienie historii operacji stosowych oraz wpływu tych operacji na dalsze przetwarzanie wejścia.

Rozszerzenie algorytmu L^* dla automatów ze stosem umożliwia analizę bardziej złożonych języków formalnych niż języki regularne. Podejście to może znaleźć zastosowanie między innymi w analizie składni języków programowania, automatycznej inferencji parserów, analizie protokołów komunikacyjnych oraz modelowaniu struktur hierarchicznych wykorzystywanych w systemach sztucznej inteligencji.

Opracowane rozwiązanie stanowi podstawę implementacji platformy badawczej przedstawionej w dalszej części pracy. Platforma umożliwia eksperymentalną analizę procesu uczenia automatów ze stosem oraz porównanie skuteczności klasycznych i rozszerzonych metod inferencji struktur formalnych.

3.6 Założenia projektowe platformy badawczej

Jednym z głównych celów niniejszej pracy jest opracowanie platformy badawczej umożliwiającej analizę oraz generowanie struktur formalnych z wykorzystaniem metod sztucznej inteligencji. Projektowana aplikacja ma stanowić środowisko eksperymentalne pozwalające na badanie procesu inferencji języków formalnych, analizę działania automatów oraz testowanie rozszerzonego algorytmu L^* przeznaczonego dla automatów ze stosem.

Podstawowym założeniem projektowym platformy jest modularność architektury systemu. Aplikacja została zaprojektowana w sposób umożliwiający niezależny rozwój poszczególnych komponentów odpowiedzialnych za analizę języków formalnych, obsługę automatów, generowanie struktur formalnych oraz integrację z modułami sztucznej inteligencji. Takie podejście pozwala na łatwe rozszerzanie systemu o nowe algorytmy oraz modele formalne bez konieczności modyfikacji całej architektury aplikacji.

Istotnym założeniem platformy jest obsługa różnych klas języków formalnych, w szczególności języków regularnych oraz języków bezkontekstowych. System powinien umożliwiać definiowanie automatów skończonych, automatów ze stosem, gramatyk formalnych oraz wyrażeń regularnych. Dzięki temu możliwe staje się przeprowadzanie eksperymentów porównawczych pomiędzy różnymi metodami reprezentacji języków formalnych.

W projektowanej platformie szczególną uwagę poświęcono implementacji mechanizmów aktywnego uczenia automatów. System ma umożliwiać zadawanie pytań o przynależność słów do języka, analizę kontrprzykładów oraz automatyczne budowanie modeli formalnych na podstawie uzyskanych odpowiedzi. Kluczowym elementem platformy jest implementacja rozszerzonego algorytmu L^* umożliwiającego analizę struktur wykorzystujących mechanizm stosowy.

Kolejnym ważnym założeniem projektowym jest możliwość wizualizacji działania automatów oraz procesu inferencji języka. Platforma powinna umożliwiać graficzną prezentację stanów automatów, przejść pomiędzy stanami, operacji wykonywanych na stosie oraz zmian zachodzących w strukturach danych wykorzystywanych przez algorytm uczenia. Wizualizacja procesu analizy ma ułatwić interpretację wyników eksperymentów oraz analizę skuteczności proponowanego rozwiązania.

System został również zaprojektowany z myślą o integracji z nowoczesnymi modelami sztucznej inteligencji. Platforma powinna umożliwiać wykorzystanie modeli językowych do wspomagania generowania struktur formalnych, automatycznej analizy składniowej oraz identyfikacji wzorców występujących w danych wejściowych. Dzięki temu możliwe staje się połączenie klasycznych metod symbolicznych z technikami uczenia maszynowego.

Istotnym wymaganiem projektowym jest również możliwość przeprowadzania eksperymentów badawczych oraz analizy wyników działania systemu. Platforma ma umożliwiać generowanie przykładowych języków formalnych, testowanie poprawności działania automatów oraz porównywanie skuteczności różnych metod inferencji. System powinien także pozwalać na zapisywanie wyników eksperymentów oraz ich dalszą analizę.

W ramach pracy założono również możliwość wykorzystania platformy jako narzędzia edukacyjnego wspomagającego naukę teorii języków formalnych oraz automatów. Interaktywna forma prezentacji struktur formalnych i procesu działania algorytmów może ułatwić zrozumienie zagadnień związanych z analizą składniową, inferencją języków oraz teorią automatów.

Opracowana platforma badawcza stanowi podstawę części implementacyjnej niniejszej pracy. Szczegółowy opis architektury systemu, zastosowanych technologii oraz działania poszczególnych modułów przedstawiono w kolejnym rozdziale.